

Day 9 — Verification Planning Materials

Refined FIFO feature list with categories and Req-IDs, entry/exit criteria for the sequence detector, and a traceability table.

Items 1–3 — Refined FIFO Feature List, Test Categories, and Req-IDs

Starting point: the Day 3 list

Day 3's original 5 features (written before any RTL existed, from the spec alone):

1. Reset initializes the FIFO to empty
2. Basic write increments count and stores data correctly
3. Basic read returns data in FIFO order
4. `full` asserts exactly when `count == DEPTH` and blocks further writes
5. Simultaneous read+write (neither empty nor full) leaves count unchanged

Refinement — what changed and why

Feature 4 was too coarse. It bundles two independently-testable behaviors: (a) the *flag* asserting correctly, and (b) a *write actually being blocked* as a consequence. You could break one without breaking the other (e.g., `full` could assert correctly but a write could still incorrectly sneak through) — so per Learning Topic 9.2's granularity guidance, I split it into two.

A missing feature, found by checking for symmetry. The original list only ever mentions `full`/write-blocking — there's no equivalent entry for `empty`/read-blocking anywhere in the Day 3 list, even though it's exactly as important and exactly as likely to break independently. This wasn't a granularity problem, it was a **gap** — I only found it by deliberately checking “is there a mirror-image feature for every one-sided one,” which is a habit worth calling out explicitly rather than silently fixing.

Feature 5 was NOT split further. I considered breaking simultaneous read+write into three (while full / while empty / while partially-filled) — this is exactly how the Day 6 testbench actually tested it — but Learning Topic 9.2 explicitly cites `data_preserved_on_simultaneous_read_write` as a *good* (not too fine) example, so I kept it as one feature. The test description for it can (and should) mention that all three sub-cases get exercised, without needing three separate Req-IDs.

Refined list — 8 features, with category and Req-ID

#	Feature	Category	Req-ID	Justification for category
1	<code>reset_initializes_fifo_to_empty</code>	RST	RST_001	Verifies startup state — the definition of a Reset/Initialization test.
2	<code>write_increments_count_and_stores_data_correctly</code>	FUNC	FUNC_001	Core intended behavior — the write-side happy path.
3	<code>read_returns_data_in_fifo_order</code>	FUNC	FUNC_002	Core intended behavior — the read-side happy path.
4	<code>full_flag_asserts_when_count_equals_depth</code>	CORNER	CORNER_001	Tests the exact boundary value (<code>count==DEPTH</code>), not general functionality.
5	<code>write_blocked_when_full</code>	NEG	NEG_001	Tests handling of an illegal input (writing with no room) — the definition of a negative test.
6	<code>empty_flag_asserts_when_count_equals_zero</code>	CORNER	CORNER_002	Boundary value at the opposite extreme (<code>count==0</code>), symmetric to #4.
7	<code>read_blocked_when_empty</code>	NEG	NEG_002	Illegal input handling (reading with nothing valid) — symmetric to #5.

#	Feature	Category	Req-ID	Justification for category
8	count_unchanged_on_simultaneous_read_write	CORNER	CORNER_003	A legal but tricky <i>interaction</i> at the edge of normal operation, not itself illegal input (so not NEG) and not the plain happy path (so not FUNC).

Item 4 — Entry and Exit Criteria: Sequence Detector

Being specific about what “done” actually means, per the assignment’s instruction — not just restating the general definitions from Learning Topic 9.5.

Entry criteria

Must all be true before verification planning/testing begins.

- **RTL is frozen:** `seq_detect_1011.sv` matches exactly what’s in `RTL_Specifications.docx` — confirmed via the Day 8 re-read pass, which found and closed the last outstanding inconsistencies (`ISSUE-D8-01`).
- **Spec is complete:** interface table, state encoding, transition table, and corner-case list are all written and verified against the actual RTL (Days 7–8).
- **Simulation environment confirmed working:** Icarus Verilog v12.0 toolchain, already used successfully for this exact design (Day 4).
- **No known open issues** remain for this design, in documentation or RTL (`ISSUE-D4-01` closed on Day 8).

Exit criteria

Specific, measurable conditions that define “done.”

- **Corner-case traceability.** All 4 corner cases identified in Day 7 have at least one passing test with a linked coverage item — full traceability, no orphaned corner case.
- **100% FSM state coverage.** All 5 states (`IDLE`, `S1`, `S10`, `S101`, `S1011`) entered at least once. Given this design only has 5 states, 100% is the right bar here — not the general “90%” guideline, which exists for much larger designs where full closure is impractical.
- **100% FSM transition coverage.** All 10 transition arcs (5 states × 2 input values) traversed at least once — not just each state visited, but every edge in the transition table actually exercised. The 10 arcs are: `IDLE→IDLE(0)`, `IDLE→S1(1)`, `S1→S10(0)`, `S1→S1(1)`, `S10→IDLE(0)`, `S10→S101(1)`, `S101→S10(0)`, `S101→S1011(1, match)`, `S1011→S10(0)`, `S1011→S1(1, overlap)` — including both self-loops (`IDLE→IDLE`, `S1→S1`).
- **Zero open P0 bugs.** For this design specifically, P0 = correct detection of `1011` (including the overlapping case) and correct reset behavior; P1 = the specific cycle-timing of `detected` (unambiguous in the Moore design: the pulse is HIGH exactly in state `S1011`; see `ISSUE-D4-01`, corrected).
- **Overlap confirmed by a dedicated test.** The overlapping-detection corner case (#1 in the Day 7 list) is confirmed by a dedicated, clearly-labeled, self-checking test — not merely by a smoke test that happens to exercise it. The `seq_detect_1011_overlap_tb` drives `1011011` (two `1011` occurrences sharing the middle `1`) and asserts exactly two detections, 3 cycles apart. Note: the Day 4 smoke vector `1101011` does **not** contain two overlapping matches — it has a single `1011` at bits 4–7 — so it cannot stand in for this criterion.

Item 5 — Traceability Table: 3 FIFO Features

Feature Name	Req-ID	Test Description	Coverage Item
full_flag_asserts_when_count_equals_depth	CORNER_001	Fill the FIFO with exactly DEPTH writes; confirm full asserts on the same cycle count reaches DEPTH, and stays HIGH until a read occurs.	Coverpoint on count, bin full = {DEPTH}.
write_blocked_when_full	NEG_001	With the FIFO full, assert wr_en for one cycle using a distinct “poison” data value; confirm count and memory contents are unchanged, and the poison value never appears in any later read.	Cross coverage: wr_en × full, bin blocked_write = {wr_en=1, full=1}.
count_unchanged_on_simultaneous_read_write	CORNER_003	Assert wr_en and rd_en together while the FIFO is neither full nor empty; confirm count is identical before and after, and both the write and read complete correctly. Also exercise the full-and-empty sub-cases of this same scenario.	Cross coverage: wr_en × rd_en × {full, empty} status, bins for simul_normal, simul_while_full, simul_while_empty.